

Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №4

По дисциплине «Логика и основы алгоритмизации в инженерных задачах»

На тему «Бинарное дерево поиска.»

Выполнили студенты группы
25BVB3:

Сергеева С.М.

Иванова С.А.

Приняли:

к.т.н., доцент Юрова О. В.

к.т.н., доцент Деев М. В.

Пенза 2026

Цель: Освоить операции над бинарным деревом поиска, реализовать поиск заданного значения, подсчёт числа его вхождений, исключение дубликатов при добавлении и оценить сложность поиска.

Ход работы.

Реализован алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве. Добавлена функция подсчёта числа вхождений заданного элемента в дерево. Внесены изменения в функцию добавления элементов для исключения добавления одинаковых символов.

Описание программы:

Поиск выполняется функцией Find, которая получает указатель на корень дерева и искомое значение. Если искомое значение больше текущего узла — поиск продолжается в левом поддереве, если меньше — в правом, если равно — узел найден. Если дошли до NULL, значит значения в дереве нет. Функция возвращает указатель на найденный узел или NULL. В меню это пункт «3 — Найти значение»: пользователь вводит число, программа выводит сообщение «найдено» или «не найдено».

Подсчёт выполняет функция CountValue. Она рекурсивно обходит всё дерево и для каждого узла прибавляет 1, если его значение совпадает с искомым, иначе 0. Результат складывается с количеством совпадений в левом и правом поддеревьях. Функция возвращает общее число вхождений. В меню это пункт «4 — Подсчитать вхождения значения». Так как дубликаты в дерево не добавляются, результат всегда будет 0 или 1, но функция корректно работает и при наличии повторов.

Случай `data == r->data` не обрабатывается — рекурсивный вызов не происходит, новый узел не создаётся, значение-дубликат игнорируется. Раньше стояло просто `else`, из-за чего равные элементы уходили в правую ветку и добавлялись повторно. Теперь дерево содержит только уникальные значения, а количество узлов равно количеству различных введённых чисел.

Сложность поиска значения в бинарном дереве поиска определяется его высотой h и в среднем составляет $O(\log n)$, где n — количество узлов. В

худшем случае, когда дерево вырождается в линейный список, поиск требует $O(n)$ операций. Таким образом, эффективность поиска напрямую зависит от степени сбалансированности дерева, которая определяется порядком добавления элементов.

```
1 - Построить дерево
2 - Вывести дерево
3 - Найти значение
4 - Подсчитать вхождения значения
0 - Выход
Введите число:
```

Рисунок 1 – Меню

```
1 - Построить дерево
2 - Вывести дерево
3 - Найти значение
4 - Подсчитать вхождения значения
0 - Выход
Введите число: 1
-1 - окончание построения дерева
Введите число: 2
Введите число: 3
Введите число: 5
Введите число: 8
Введите число: -1
Построение дерева окончено
Нажмите Enter для возврата в меню
```

Рисунок 2 – построение дерева

```
1 - Построить дерево
2 - Вывести дерево
3 - Найти значение
4 - Подсчитать вхождения значения
0 - Выход
Введите число: 3
Введите значение для поиска: 8
Значение 8 найдено
Нажмите Enter для возврата в меню
```

Рисунок 3 – Поиск значения

```
1 - Построить дерево
2 - Вывести дерево
3 - Найти значение
4 - Подсчитать вхождения значения
0 - Выход
Введите число: 4
Введите значение для подсчёта: 2
Значение 2 встречается 1 раз

Нажмите Enter для возврата в меню
```

Рисунок 4 – Подсчет вхождения значений

```
1 - Построить дерево
2 - Вывести дерево
3 - Найти значение
4 - Подсчитать вхождения значения
0 - Выход
Введите число: 0

C:\Users\osio\Desktop\src\laba4\x64\Debug\laba4.exe (процесс 15116) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 5 – Завершение программы

Вывод: В ходе лабораторной работы мы освоили операции над бинарным деревом поиска, реализовали поиск заданного значения, подсчёт числа его вхождений, исключение дубликатов при добавлении и оценили сложность поиска.